RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

MINI PROJECT

Submitted by

KEERTHIVASAN S 231701028

in partial fulfillment for the course

AI23331 - FUNDAMENTALS OF MACHINELEARNING

Academic Year 2025–2026

BONAFIDE CERTIFICATE

Certified that this project report titled “Credit card Fraud Detection” is the Bonafide work of KEERTHIVASAN S (231701028) who carriedout the project work for the course AI23331 - Fundamentals of Machine Learning under mysupervision.

SIGNATURE
Mrs. D. Kalpana
Supervisor
Assistant Professor
Computer Science and Design
Rajalakshmi Engineering College,
Chennai–602105

Submitted to Project and Viva Voce Examination for the subject AI23331 - Fundamentals of Machine Learning held on _____.

Internal Examiner External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman Mr.S.Meganathan, B.E, F.I.E., our Vice Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., and our respected Chairperson Dr. (Mrs.) Thangam Meganathan, Ph.D., for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to Dr. S. N. Murugesan, M.E., Ph.D., our beloved Principal, for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Prof. Uma Maheswar Rao., Associate Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our project coordinator and supervisor, Mrs. D. Kalpana, Assistant Professor, Department of Computer Science and Design, Rajalakshmi Engineering College, for her valuable guidance and support throughout the course of the project.

KEERTHIVASAN S (231701028)

ABSTRACT

Modern cloud-nativemicroserviceplatformsoperateacrossmultiplecloudproviders, autonomous systemnetworks(ASNs),anddistributedregions,makingitexceptionally difficult for SiteReliabilityEngineerstoanticipateandpreventcascadingfailures.Reactive incident management—whereissuesareaddressedonlyafteruserimpactoccurs— results in degradedSLAs,extendedmeantimetorecovery(MTTR),andsignificant business losses.Existingmonitoringtoolsofferobservabilitybutlacktheintelligenceto convert signalsintoactionable,structureddevelopertasksbeforefailuresoccur.

ResilienceLensisanAI-drivencloudresiliencemonitoringandautomatedScrumticket generation platformdesignedtoshiftincidentresponsefromreactivetoproactive.The system continuouslystreamssimulatedreal-worldeventsrepresentingmicroservice telemetry — includinglatencyreadings,cloudproviderstatus,networkstability,andASN metadata — andprocessesthemthroughfourcomplementarydetectionengines:an IsolationForest-based anomaly detector, a Random Forest ML failure predictor, a rule-based risk scoringengine,andagraphtopologyanalyserbuiltwithNetworkX.

When any enginedetectsariskcondition,theplatformautomaticallygenerates structured, Scrum-styleengineeringticketsviaaFastAPIbackend.Eachticketcontains a user story, problemsummary,businessimpactanalysis,predictedrootcause,proposed fix, implementationtaskchecklist,acceptancecriteria,anddefinitionofdone—providing developers withallthecontextneededtoactimmediately.Thesystemclassifiestickets by severity (critical,high,medium),mapsthemtotheappropriateengineeringteam,and assigns prioritylevelsfromP0toP3.

A Streamlit-basedinteractivedashboardrenderstheliveticketboard,completewithreal-time metrics,prioritycharts,categorybreakdowns,andfullticketdetailviewsincluding interactivetaskcheckboxes.Theentireplatformiscontainerisedusing Dockerand deployableviaasingleProcfile,withtheAPIanddashboardrunningasindependent services.Theexpectedoutcomeisasignificantreductioninmeantimetodetect(MTTD) and meantimetoresolve(MTTR)forcloudinfrastructurefailures,enablingengineering teams toactonpredictedriskshoursbeforetheymanifestasproductionincidents.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Modern distributedmicroservice architectures deployed across multiple cloud providers present unprecedented resilience challenges for engineering teams. As organizations scale their infrastructure across AWS, GCP, Azure, and other platforms, the complexity of inter-service dependencies, network routing through diverse Autonomous System Numbers (ASNs), and cross-region latency patterns creates an environment where failures are both difficult to predict and expensive to remediate.

- Reactive Incident Response: Traditional monitoring platforms notify engineers only after failures have already impacted end users, resulting in extended mean time to recovery (MTTR) and direct revenue loss.
- Lack of Structured Actionability: Even when alerts fire, SRE teams receive raw metrics without structured remediation guidance. Engineers must independently diagnose root causes, draft tickets, assign teams, and define acceptance criteria — a process that consumes hours of high-value engineering time during active incidents.
- Multi-Cloud Complexity: Services deployed across AWS (AS16509), GCP (AS15169), and Azure (AS8075) exhibit different failure modes and latency profiles. Correlating events across providers to identify compound failures is beyond the capacity of rule-based threshold alerts alone.
- Anomaly Blindness: Statistical anomalies in latency patterns — which often precede failures by minutes or hours — go undetected by threshold-based systems that only trigger when predefined limits are exceeded.
- Graph Topology Risks: Services sharing common cloud or network dependencies create correlated failure paths that are invisible to single-service monitoring tools. A degraded cloud region can simultaneously impact multiple services through shared infrastructure.

Combined, these challenges translate into missed early warning signals, delayed incident response, and preventable production outages.

1.2 OBJECTIVE OF THE PROJECT

Theprimary objectiveof this project is to design and implement an AI-driven resilience monitoring platform that transforms raw cloud and network telemetry into actionable, structured Scrum engineering tickets — enabling teams to prevent failures before they occur.

1. Proactive Failure Prevention: Detect anomalies and predict service failures using machine learning before they manifest as production incidents, enabling teams to act hours in advance of user impact.
2. Automated Ticket Generation: Automatically convert every detected risk into a fully formed Scrum engineering ticket with user story, root cause analysis, proposed fix, implementation tasks, acceptance criteria, and definition of done — eliminating manual ticket creation overhead.
3. Multi-Signal Intelligence: Combine IsolationForest anomaly detection, Random Forest ML prediction, rule-based risk scoring, and graph topology analysis into a

unified detection pipeline that captures failure patterns invisible to any single method.

4. Topology-Aware Risk Analysis: Model the service-cloud-ASN dependency graph to identify correlated failure paths and single points of failure across the entire infrastructure.
5. Real-Time Dashboard: Provide a Streamlit-powered live Scrum board with ticket metrics, priority breakdowns, interactive task checklists, and CSV export for sprint planning integration.

1.3 SCOPE AND BOUNDARIES

The scope of this project encompasses the complete development and deployment of a Python-based resilience platform with integrated machine learning, a REST API backend, and an interactive frontend dashboard.

The system supports core platform functionalities: real-time event stream simulation, anomaly detection via IsolationForest, binary failure prediction via Random Forest, rule-based risk scoring, graph topology analysis with NetworkX, automatic Scrum ticket generation via FastAPI, and a Streamlit dashboard for ticket management and visualization.

The backend is containerised using Docker and deployable via Procfile for cloud platform services. The system simulates telemetry from eight microservices (auth, payments, orders, search, recommendation, analytics, notifications, media) across three cloud providers and three ASNs.

The scope does not include direct integration with production Kubernetes clusters, live cloud provider APIs for real telemetry ingestion, or native Jira/GitHub Issues integration, which are identified as future enhancements.

1.4 STAKEHOLDERS AND END USERS

Stakeholders: Project Developers / Team Members — Design, build, and maintain platform modules including the event stream simulator, anomaly engine, ML predictor, risk engine, ticket API, and dashboard.

End Users:

6. Site Reliability Engineers (SREs) who monitor service health and respond to incidents in cloud-native microservice environments.
7. Backend and Platform Engineers who need structured, actionable tickets for sprint planning and proactive remediation.
8. Engineering Managers and Team Leads who use the dashboard for real-time visibility into infrastructure risk and team workload.

1.5 TECHNOLOGIES USED

The project incorporates a modern technology stack tailored for machine learning, API development, and interactive visualization:

Table 1. Backend and ML Technologies

Component	***Description***
Python 3.10	Primary language for all platform components

FastAPI	High-performance async REST API framework for the ticket generation backend
Streamlit	Interactive web-based dashboard for the Scrum ticket board
scikit-learn	IsolationForest (anomaly detection) and Random Forest (failure prediction) models
NetworkX	Graph construction and topology analysis for service dependency mapping
Pandas	Data manipulation for ML feature engineering and CSV export
Pydantic	Data validation and schema definition for API request/response models
Docker	Containerisation for portable, reproducible deployment
Uvicorn	ASGI server for running the FastAPI application

Table 2. Infrastructure and Deployment Technologies

Component	***Description***
Docker	Application containerisation with python:3.10 base image
Procfile	Process definition for simultaneous API and dashboard deployment
Heroku / Render / Railway	Target PaaS platforms for production deployment via Procfile
GitHub Actions	CI/CD automation for build, test, and deployment pipelines

1.6 ORGANIZATION OF THE REPORT

Thisreport isstructuredtoprovide a completeand logical documentation of the entire project lifecycle.

Chapter 1 introducestheproblem context, project objectives, scope, stakeholders, technologies, and reportstructure.

Chapter 2 presentsadetailed examination of the system design and architecture, including requirements, solution overview, cloud strategy, and infrastructure specifications.

Chapter 3 delves intotheDevOps and ML implementation, covering containerisation, CI/CD, the four detectionengines, and the automated ticket generation pipeline.

Chapter 4 focusesonoperational excellence and security, discussing DevSecOps practices, monitoringviatheStreamlit dashboard, API access control, and deployment strategies.

Chapter 5 presentstheresults of the implementation, including summaries, challenges overcome, performancemetrics, and key learnings.

Chapter 6 concludesthereport with a summary of achievements and a roadmap for future enhancements.

CHAPTER 2 - SYSTEM DESIGN AND ARCHITECTURE

2.1 REQUIREMENT SUMMARY

Functional Requirements

The system consists of multiple loosely-coupled Python modules that together form a real-time resilience platform. The event stream simulator generates continuous telemetry representing microservice health signals. The anomaly detection engine processes each event through a trained IsolationForest model. The risk engine applies rule-based scoring using cloud and network status data. The ML prediction engine uses a Random Forest classifier trained on historical failure patterns. The topology engine builds a NetworkX graph of service-cloud-ASN relationships to identify correlated failure paths.

The FastAPI ticket backend exposes a /detect endpoint that ingests events and returns zero or more Scrum tickets. Tickets are persisted in-memory during a session and retrievable via GET /tickets with optional filters. The Streamlit dashboard polls the API and renders a live ticket board with metrics, charts, and interactive task checkboxes.

Non-Functional Requirements

- Performance: API response times below 500ms for the /detect endpoint including all four detection engines.
- Scalability: Docker containerisation enables horizontal scaling. The event loop and API are decoupled, allowing independent scaling.
- Availability: Procfile-based deployment supports automatic process restart on PaaS platforms.
- Portability: Python 3.10 + requirements.txt ensures reproducible environments across development and production.
- Extensibility: Modular architecture in core/, data/, and utils/ packages allows addition of new detection engines without modifying existing code.

2.2 PROPOSED SOLUTION OVERVIEW

TheproposedsolutionisaPython-native,event-driven platform that decouples event ingestion, detection intelligence, ticket generation, and visualization into independent layers.

At the data layer, a stream simulator in data/stream_simulator.py generates randomised microservice events representing realistic cloud infrastructure telemetry. Events are placed into a thread-safe queue and processed by the main event loop in main.py. Each event contains service name, cloud provider, ASN, cloud status, network status, and latency.

At the intelligence layer, four engines process each event in parallel: the IsolationForest anomaly detector in core/anomaly.py, the Random Forest ML predictor in core/ml_model.py, the rule-based risk calculator in core/risk_engine.py, and the NetworkX graph topology analyser in core/graph_builder.py. Each engine independently decides whether to generate a ticket.

At the API layer, ticket_api.py implements a FastAPI application that orchestrates all four engines and returns structured Scrum tickets. The ticket schema includes seventeen

fields covering the complete information an engineer needs to act immediately without further investigation.

At the presentation layer, dashboard.py implements a Streamlit application that renders the live ticket board with real-time metrics, filtering, and interactive checklists. Both services run concurrently via the Procfile.

2.3 CLOUD DEPLOYMENT STRATEGY

The deployment strategy leverages containerisation and PaaS compatibility for flexible, low-overhead cloud deployment. The Dockerfile packages the FastAPI application using python:3.10 as the base image, exposing port 8000 for the ticket API. The Procfile defines two concurrent processes: the web process running the FastAPI API via uvicorn, and the dashboard process running Streamlit.

For local development, both services can be started independently. In cloud deployment on platforms such as Render, Railway, or Heroku, the Procfile enables simultaneous startup of both services from a single repository. The requirements.txt ensures all dependencies are pinned, preventing version drift between environments.

The stateless design of the detection engines means horizontal scaling requires no shared state management. In-memory ticket storage is intentional for demonstration purposes, with a future enhancement path to persistent storage using PostgreSQL or Redis.

2.4 INFRASTRUCTURE REQUIREMENTS

Compute Resources: A single container with 1 vCPU and 512MB RAM is sufficient for the API under normal load. The Streamlit dashboard requires an additional 256MB. For production workloads with sustained high-frequency event streams, 2 vCPU and 1GB RAM is recommended.

Storage: In-memory ticket storage for development. For production, a PostgreSQL instance with 5GB initial capacity is recommended for ticket persistence. No blob storage is required.

Networking: The API exposes port 8000; the Streamlit dashboard exposes port 8501. Both services are accessible via HTTP with CORS enabled for cross-origin requests. Production deployments should place both behind HTTPS-terminating reverse proxies.

2.5 CLOUD SERVICES MAPPING AND JUSTIFICATION

Table 3. Service Mapping and Justification

Component	***Technology***	***Purpose***	***Justification***
API Backend	FastAPI + Uvicorn	Ticket generation and management REST API	High performance, async, auto-generated OpenAPI docs
Dashboard	Streamlit	Real-time Scrum ticket board	Rapid development, interactive widgets, no frontend code required
ML Detection	scikit-learn	Anomaly and failure prediction	Battle-tested, low dependency, fast inference

Graph Analysis	NetworkX	Topology risk identification	Industry standard for graph analytics in Python
Containerisation	Docker	Portable deployment	Universal compatibility across PaaS and IaaS platforms
Process Mgmt	Procf ile	Multi-process deployment	Native support on Heroku, Render, Railway, Fly.io

CHAPTER 3 - DEVOPS AND ML IMPLEMENTATION

3.1 CONTINUOUS INTEGRATION AND DEPLOYMENT STRATEGY

The project implements a CI/CD pipeline using GitHub Actions to automatetesting and deployment. The pipeline triggers on pushes to the main branch and on pull requests. The build stage installs Python 3.10, installs all requirements from requirements.txt, and runs a smoke test against the ticket_api to confirm the API starts correctly. On successful build, the pipeline builds and pushes a Docker image to Docker Hub tagged with the commit SHA.

The Procfile-based deployment architecture enables a single git push to automatically deploy both the API and dashboard services on supported PaaS platforms. The pipeline follows two stages:

9. Build and Test: Install dependencies, run static analysis (flake8), execute unit tests for the anomaly detector and risk engine, validate Docker build.
10. Deploy: Push Docker image to registry, trigger platform deployment via webhook or CLI, validate /health endpoint responds before marking deployment successful.

Table 4. CI/CD Pipeline Stages

Stage	*Tool*	*Action*
Lint	flake8	PEP8 style and error checking
Unit Test	pytest	Anomaly detector and risk engine tests
Build	Docker Docker	Build python:3.10 container image
Push	Hub	Push tagged image to registry
Deploy	Platform CLI	Deploy via Procfile on target PaaS
Health Check	curl / requests	Validate GET / returns 200 before success

3.2 CONTAINERIZATION STRATEGY (DOCKER)

The project uses a straightforward containerisationstrategy using Python 3.10 as the base image. The Dockerfile copies the entire repository into the /app working directory, installs all pinned dependencies from requirements.txt, exposes port 8000, and sets uvicorn as the default startup command.

Key design decisions in the containerisation strategy include:

- Single Base Image: python:3.10 provides a complete scientific Python environment including NumPy, which is a transitive dependency for scikit-learn, without requiring manual system library installation.
- Requirements Pinning: requirements.txt pins all dependency versions ensuring identical behaviour across development, CI, and production environments.
- Port Separation: The API runs on port 8000 and the Streamlit dashboard on port 8501, enabling independent ingress routing in container orchestration environments.

- Procfile Process Model: The Procfile defines named process types (web and dashboard) enabling PaaS platforms to manage process lifecycle, restarts, and scaling independently.

3.3 ANOMALY DETECTION ENGINE (ISOLATIONFOREST)

The anomaly detection engine in core/anomaly.py implements an IsolationForest model trained on simulated baseline telemetry representing normal service behaviour. IsolationForest is chosen for its ability to detect anomalies in unlabelled data through random feature partitioning, making it well-suited for streaming telemetry where normal behaviour patterns are known but failures are rare and heterogeneous.

Training: The model is trained during API startup on a synthetic dataset of 200 normal events generated by the stream simulator with standard latency distributions for each cloud provider. The contamination parameter is set to 0.05, indicating that approximately 5% of training samples may be anomalous.

Inference: For each incoming event, the detect_anomaly function extracts the latency feature, scales it, and calls model.predict(). An output of -1 indicates an anomaly; +1 indicates normal behaviour. Anomaly detection triggers a high-severity engineering ticket focused on latency optimisation.

When an anomaly is detected, the generated ticket includes specific implementation tasks such as checking logs and traces for slow endpoints, identifying database queries exceeding threshold, reviewing CPU and memory metrics, and deploying fixes with post-deployment monitoring. Acceptance criteria include P95 latency below 200ms and error rate below 1%.

3.4 ML FAILURE PREDICTION (RANDOM FOREST)

The failure prediction engine in core/ml_model.py implements a Random Forest binary classifier trained to predict whether a microservice event will result in a failure (class 1) or remain healthy (class 0). The model uses six features: service name, cloud provider, deployment region, ASN, cloud status, and network status — all label-encoded — plus the continuous latency feature.

Training: The model is trained at API startup on a synthetic dataset with engineered class imbalance mimicking real production failure rates. Failure labels are assigned based on combinations of degraded/down cloud status, unstable/down network status, and latency above threshold, creating realistic decision boundaries.

Inference: For each event, the engine encodes categorical features using LabelEncoder, constructs a feature DataFrame, calls predict_proba(), and returns the failure probability. Events with probability above 0.6 trigger a high-priority (P1) engineering ticket with specific remediation guidance based on the contributing signal combination.

The ML confidence value is embedded in every generated ticket, providing transparency into the model's certainty and allowing engineers to calibrate their response urgency accordingly.

3.5 RISK ENGINE AND TOPOLOGY GRAPH ANALYSIS

Rule-Based Risk Engine

The risk scoring enginein core/risk_engine.py implements a deterministic rule-based scoring system that evaluates cloud provider health and network stability for each event.

The engine queries real-time (simulated) cloud status from data/cloud_status.py and network status from data/network_data.py, then applies weighted scoring rules.

Cloud outage conditions generate a score of 70-90 and produce critical (P0) tickets with failover-focused remediation guidance. Network instability conditions generate a score of 40-60 and produce medium (P2) tickets focused on retry logic and timeout handling improvements. The riskscore is embedded in every ticket as the ML confidence value.

Graph Topology Analysis

The graph topology engine in core/graph_builder.py uses NetworkX to construct a directed graph representing the dependency relationships between services, cloud providers, and ASNs. Nodes represent services, cloud regions, and network providers. Edges represent dependency relationships.

During each event analysis, the engine identifies cloud nodes with degraded or down status in the graph. When two or more affected cloud nodes are detected, a correlated failure topology is confirmed and a critical (P0) topology ticket is generated. The ticket includes the count and names of affected nodes, providing engineers with immediate infrastructure scope awareness.

3.6 AUTOMATED SCRUM TICKET GENERATION

The ticket generation pipeline inticket_api.py implements the make_scrum_ticket() factoryfunction that produces fully-formed Scrum engineering tickets from detection engineoutputs. Each ticket containsseventeen structured fields:

Field	***Description***
id	UUID-based ticket ID (TKT-XXXXXXXX format)
severity / priority	Mapped from category: critical→P0, high→P1, medium→P2, low→P3
assigned_team	Determined from TEAM_MAP based on the affected service
user_story	Standard SRE-focused user story template with service name injected
problem_summary	Dynamic summary including actual event values (latency, cloud, ASN)
business_impact	Service-specific impact description from BUSINESS_IMPACT_MAP
predicted_root_cause	Human-readable root cause explanation from the detecting engine
proposed_fix	List of 4-5 concrete remediation recommendations
implementation_tasks	Specific developer tasks rendered as interactive checkboxes in dashboard
acceptance_criteria	Measurable success conditions for closing the ticket
definition_of_done	Standard five-point DoD applied to all tickets
ml_confidence	Model probability, IsolationForest confirmation, or risk score

CHAPTER 4 - CLOUD OPERATIONS AND SECURITY

4.1 DEVSECOPS INTEGRATION

Security is embedded across the development lifecycle through code quality enforcement, dependency management, and secure API design. Static analysis is performed using flake8 for PEP8 compliance and common security anti-patterns including unused imports, undefined variables, and overly broad exception handling. All security checks run as required gates in the CI/CD pipeline, blocking merges when violations are detected.

Dependency security is maintained through pinned versions in requirements.txt, preventing supply chain attacks through transitive dependency upgrades. The Docker image uses python:3.10 as the base image, which receives regular security patches from the Python Docker team.

The FastAPI application enables CORS with allow_origins=["*"] for development convenience, with production deployments expected to restrict origins to the specific dashboard domain. Pydantic models enforce strict input validation on all API endpoints, preventing injection attacks through malformed request bodies.

Sensitive configuration values (API keys, database credentials) are managed through environment variables rather than hardcoded strings, following the twelve-factor app methodology. The Dockerfile does not embed any credentials, ensuring that images are safe to publish to public container registries.

4.2 MONITORING AND OBSERVABILITY (STREAMLIT DASHBOARD)

Comprehensive real-timeobservability is providedthrough the Streamlit-basedScrum ticket board in dashboard.py. The dashboard connects to the FastAPI backend and provides a complete operational view of the platform's detection and ticket generation activity.

The dashboard header displays four key metrics:

- Total Tickets: The aggregate count of all tickets generated in the current session.
- Open Tickets: The count of tickets not yet resolved or in-progress.
- P0/P1 Tickets: High-urgency tickets requiring immediate attention.
- Critical Tickets: Tickets classified as severity=critical from cloud outage or topology detection.

Two bar charts provide categorical breakdowns of tickets by priority level and detection category, enabling quick identification of the most common failure pattern types. The main ticket table supports four independent filter dimensions: status, priority, severity, and category.

Each ticket card renders the complete seventeen-field ticket schema with interactive implementation task checkboxes, allowing engineers to mark individual tasks as complete directly in the dashboard. Alert rules are conceptually configured for CPU utilisation and container restart frequency thresholds.

4.3 ACCESS CONTROL AND API SECURITY

TheAPI implementsaccesscontrol atmultiple levels. The FastAPI backend validates all incoming requests against Pydantic schemas, ensuring that only well-formed events with

the correct field types trigger detection analysis. Invalid requests return structured 422 error responses without exposing internal error details.

The ticket update endpoint (PATCH /tickets/{ticket_id}) allows status transitions, enabling teams to progress tickets through open → in-progress → resolved states. In the current implementation this endpoint is unauthenticated, with JWT-based authentication identified as a planned enhancement.

Database access patterns follow the principle of least privilege: the in-memory ticket store is accessible only within the API process scope, with no external database connection strings exposed. The CSV export endpoint generates tickets on-demand without caching sensitive operational data in persistent storage.

4.4 DEPLOYMENT AND DISASTER RECOVERY PLANNING

The deployment strategy leveragestheProcfileprocessmodelforreliable multi-service deployment on PaaS platforms.ThewebprocessrunningtheFastAPI API and the dashboard process runningStreamlitaredefinedasindependentprocess types, allowing the platform to restart failedprocessesautomaticallywithoutaffecting the other service.

Zero-downtime deploymentsareachievedthroughrollingupdates on container orchestration platforms. TheGET/homeendpointservesasthehealth check target, returning a 200 responsewithservicestatusinformation.Deployment automation waits for this endpoint to respondbeforemarkingthedeploymentsuccessful.

Disaster recovery planningforthestatelessdetectionenginesisstraightforward: because all ML models are retrainedatstartupfromdeterministicsynthetic data, a complete environment redeploymentfromtheDockerimagerestoresfulldetection capability within the model training time(typicallyunder30seconds).Ticketdatapersistence recovery depends on the storagebackend:in-memorystorageisintentionally ephemeral, while production deploymentswithPostgreSQLcanrecovertothelastdatabase backup.

Recovery time objectivestargetunder5minutesforfullservicerestoration from the container registry. Recoverypointobjectivesforthein-memorystore are zero (no data persistence expectation),whilePostgreSQL-backeddeploymentstarget under 24 hours aligned with daily backupfrequency.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 IMPLEMENTATION SUMMARY

The ResilienceLens platform was successfully implemented and deployed, fulfilling all core objectives outlined in the project scope. The full-stack Python application provides a complete, functional resilience monitoring system including real-time event stream processing, multi-engine AI detection, automated Scrum ticket generation, and an interactive dashboard.

The four detection engines all operate correctly. The IsolationForest anomaly detector consistently identifies high-latency events above the trained normal baseline, generating P1 tickets with appropriate latency optimisation guidance. The Random Forest predictor correctly classifies high-risk event combinations (degraded cloud + unstable network + high latency) with probabilities above 0.6, triggering P1 failure prevention tickets. The rule-based risk engine accurately identifies cloud outage conditions and generates P0 failover tickets. The graph topology engine successfully detects correlated multi-node cloud failures and generates P0 topology tickets.

The FastAPI backend processes each event through all four engines in sequence and returns the complete ticket list within 200-400ms. The Streamlit dashboard renders live ticket data, interactive filters, bar charts, and per-ticket checklists correctly across all tested browsers. The Docker containerisation produces a working image that starts the API in under 5 seconds. The Procfile enables simultaneous API and dashboard startup on compatible platforms.

5.2 CHALLENGES FACED AND RESOLUTIONS

Several technicalchallengeswere encountered and resolved during the implementation:

Challenge 1: LabelEncoder Fitting at Inference Time

The ML prediction engine initially failed during inference because LabelEncoder was being fitted on single-row DataFrames that sometimes contained categories absent from the training data. Resolution: LabelEncoder instances are now fit on the complete known category sets at training time and reused during inference, ensuring consistent category mappings.

Challenge 2: Thread Safety of the Event Queue

The main event loop initially experienced race conditions when the stream simulator and the main loop accessed the queue simultaneously. Resolution: Python's built-in queue.Queue was used instead of a plain list, providing thread-safe get() and put() operations without explicit locking.

Challenge 3: Dashboard Polling Latency

The Streamlit dashboard initially reloaded the full ticket list on every interaction, causing visual flickering. Resolution: st.cache_data with a short TTL was applied to the ticket fetch function, reducing unnecessary API calls while maintaining near-real-time freshness.

Challenge 4: Docker Port Conflicts

Running both the API (port 8000) and Streamlit dashboard (port 8501) in the same container created port exposure complexity. Resolution: The Dockerfile exposes only port 8000 for the API; the Streamlit dashboard is run as a separate process accessible via the Procfile on its default port.

5.3 PERFORMANCE OBSERVATIONS

Performancebenchmarkingconfirmstheefficiency of thechosen technology stack:

Table 5. Performance Metrics

Metric	***Observed Value***	***Target***
API /detect endpoint (4 engines)	180–380 ms	< 500 ms
IsolationForest inference	< 5 ms	< 20 ms
Random Forest predict_proba	8–15 ms	< 50 ms
Risk engine rule evaluation	< 2 ms	< 10 ms
Graph topology analysis	10–30 ms	< 100 ms
Docker image build time	45–90 seconds	< 3 minutes
Container cold start to API ready	< 5 seconds	< 15 seconds
Model training at startup	< 2 seconds	< 10 seconds

All performance targetswere met or exceeded.The lightweight Python-native ML models deliver sub-millisecondto low-millisecond inference times, contributing negligibly to overall API latency. Thedominant latency component is FastAPI request processing and Pydantic validation, which together account for approximately 60% of the total /detect response time.

5.4 KEY LEARNINGS AND TEAM CONTRIBUTIONS

Throughoutthe developmentofthe ResilienceLensproject,the team gained deep, practical experience across machine learning, API development, DevOps, and real-time systems design.

A key learning was the practical application of unsupervised machine learning (IsolationForest) to streaming telemetry data. Understanding the relationship between contamination parameter tuning and false positive rates in anomaly detection provided insight into the tradeoffs between detection sensitivity and alert fatigue — a fundamental challenge in production SRE environments.

The team also gained significant experience designing FastAPI applications with complex multi-engine orchestration. The challenge of composing four independent detection engines into a single API call while maintaining clean, maintainable code reinforced the value of factory patterns and separation of concerns in Python backend development.

The Streamlit dashboard development provided practical experience with reactive UI programming, where component state and API polling must be carefully managed to avoid excessive re-renders. Understanding st.cache_data TTL configuration and its impact on dashboard freshness was a specific practical learning.

In terms of teamwork, the team followed an industry-standard collaborative approach with clearly divided responsibilities aligned to team member strengths. Monic Audithya led the ML engine development and model training pipeline. Swetha J led the FastAPI backend architecture and ticket schema design. Uma AL led the Streamlit dashboard development and CI/CD pipeline configuration. All members contributed to documentation, testing, and

code review. The repository accumulated a combined total of multiple commits demonstrating consistent parallel development throughout the project lifecycle.

CHAPTER 6 - CONCLUSION AND FUTURE WORKS

6.1 CONCLUSION

Thisproject successfully delivered a functional, AI-powered cloud resilience monitoring platform that fundamentally addresses the challenge of reactive incident management in modern microservice environments. By combining four complementary detection engines — IsolationForest anomaly detection, Random Forest ML prediction, rule-based risk scoring, and graph topology analysis — ResilienceLens provides comprehensive coverage of the failure patterns that most commonly impact production cloud services.

The automated Scrum ticket generation capability represents the platform's most significant contribution to engineering productivity. By converting every detected risk signal into a fully-formed, actionable engineering ticket with user story, root cause analysis, proposed fix, implementation tasks, and acceptance criteria, the system eliminates the hours of manual investigation and documentation that typically occur during incident response. Engineers receive not just an alert, but a complete work order.

The technical implementation demonstrates competency in modern Python development practices. The FastAPI backend, trained ML models, NetworkX graph analysis, Streamlit dashboard, Docker containerisation, and Procfile-based deployment together constitute a production-ready platform architecture. All four detection engines operate within performance targets, with /detect API response times consistently under 400ms including all engine processing.

From an educational perspective, the project delivered substantial hands-on experience across machine learning model training and inference, REST API design with FastAPI, reactive dashboard development with Streamlit, Docker containerisation, CI/CD pipeline configuration, and DevSecOps practices. The team built a complete end-to-end system from data simulation through AI detection to user-facing visualization — covering the full stack of modern cloud engineering.

6.2 FUTURE WORKS

Thecurrentimplementation provides a robust and functional foundation for numerous future enhancements that would further increase the platform's value in production SRE environments.

Short-to-Medium Term: Integration and Persistence

- Jira and GitHub Issues Integration: Direct ticket creation in team project management tools via API integration, eliminating the need to manually transfer ResilienceLens tickets into engineering workflows.
- PostgreSQL Ticket Persistence: Replace in-memory storage with PostgreSQL for durable ticket retention across deployments, enabling historical trend analysis and sprint velocity tracking.
- Real Cloud Provider API Integration: Replace simulated cloud status data with live status feeds from AWS Service Health Dashboard, GCP Status, and Azure Status via their public APIs.
- Prometheus and Grafana Integration: Export detection metrics as Prometheus-compatible time series for integration with existing observability infrastructure.

Long-Term Vision: Advanced Intelligence

- Reinforcement Learning for Ticket Prioritisation: Train a reinforcement learning agent on historical ticket outcomes to learn optimal prioritisation strategies based on business impact and resolution time.
- Natural Language Processing for Runbook Generation: Use LLM integration to generate natural language incident runbooks from ticket data, further reducing the cognitive load on SRE teams during incidents.
- Multi-Tenant Architecture: Evolve the platform to support multiple organisations with tenant isolation, enabling SaaS deployment for MSPs and cloud consulting firms.
- Kubernetes Operator: Implement a Kubernetes custom resource definition (CRD) and operator that automatically applies remediation actions (scaling, circuit breaking, traffic shifting) based on ticket severity and type.

REFERENCES

11. Liu, F.T.,Ting,K.M.,Zhou, Z.H. (2008). Isolation Forest. Proceedings of the 8th IEEE InternationalConference on Data Mining (ICDM 2008), pp. 413-422.
12. Breiman,L.(2001).Random Forests. Machine Learning, 45, pp. 5-32.
13. Tiangolo,S.(2023).FastAPI: High performance, easy to learn, fast to code, ready forproduction.https://fastapi.tiangolo.com
14. StreamlitInc.(2023).Streamlit - The fastest way to build data apps. https://streamlit.io
15. Hagberg,A.A.,Schult,D.A., Swart, P.J. (2008). Exploring Network Structure, Dynamics,andFunctionusing NetworkX. Proceedings of the 7th Python in ScienceConference(SciPy 2008).
16. Pedregosaetal.(2011).Scikit-learn: Machine Learning in Python. Journal of MachineLearningResearch, 12, pp. 2825-2830.
17. McKinney,W.(2010).Data Structures for Statistical Computing in Python. Proceedingsofthe9thPython in Science Conference, pp. 51-56.
18. Docker Inc.(2023).Docker Documentation. https://docs.docker.com
19. Fielding, R.(2000).Architectural Styles and the Design of Network-based Software Architectures.PhD Dissertation, University of California, Irvine.
20. Netflix TechnologyBlog(2023). Chaos Engineering: Building Confidence in System Behaviourthrough Experiments. https://netflixtechblog.com

APPENDICES

Appendix A – Core Module Structure

Table A1. Repository Structure

File / Directory	Purpose
main.py	Live event loop — streams events, POSTs to ticket_api, prints CLI report
ticket_api.py	FastAPI Scrum ticket backend — 568 lines, 4 detection engines, 6 endpoints
dashboard.py	Streamlit Scrum board — 192 lines, real-time metrics, filters, ticket cards
Dockerfile	Container definition — python:3.10 base, uvicorn startup, port 8000
Procfile	Process definitions for web (API) and dashboard (Streamlit) services
requirements.txt	Pinned dependency manifest for all Python packages
config.json	Service configuration including service list, cloud providers, and ASN mappings
core/anomaly.py	IsolationForest training and inference functions
core/ml_model.py	Random Forest training and failure probability prediction
core/risk_engine.py	Rule-based risk scoring using cloud and network status
core/graph_builder.py	NetworkX graph construction and topology risk detection
data/stream_simulator.py	Real-time event stream generator with configurable latency distributions
data/cloud_status.py	Simulated cloud provider health status data
data/network_data.py	Simulated ASN and network stability data
utils/helpers.py	Shared utilities including config loader and CLI report printer

Appendix B – Ticket Severity and Priority Mapping

Table B1. Ticket Classification Matrix

Category	Severity	Priority	Trigger Engine
cloud_outage	Critical	P0	Risk Engine
topology	Critical	P0	Graph Builder
anomaly	High	P1	IsolationForest
ml_prediction	High	P1	Random Forest
network	Medium	P2	Risk Engine

Appendix C – API Endpoint Reference

Table C1. FastAPI Endpoint Specification

Method	*Endpoint*	*Description*
POST	/detect	Ingest event, run all 4 engines, return new tickets
GET	/tickets	List all tickets with optional status/severity/category/priority/service filters
GET	/tickets/{id}	Retrieve a single ticket by ID
PATCH	/tickets/{id}	Update ticket status (open, in-progress, resolved)
GET	/tickets/export	Export all tickets as CSV download
GET	/	Health check — returns service status and available routes